R Programming Lab File

Name : Pushpesh Pant
Roll no : 25/SCA/MCAN/052
Course : MCA - ’A’

Programs

21-Jan-2026

```
# Q1: Create a vector of first 50 even numbers
even_numbers <- seq(2, 100, by = 2)
print(even_numbers)

# Q2: Create & display a vector of first 50 odd numbers
odd_numbers <- seq(1, 99, by = 2)
print(odd_numbers)

# Q3: Create a vector with 12 integers and convert it into a 4x3 matrix
vec <- 1:12
B <- matrix(vec, nrow = 4, ncol = 3)
print(B)

# Q4: Add 2 vectors of integer type and length 3
v1 <- c(2L, 4L, 6L)
v2 <- c(1L, 3L, 5L)
sum_vector <- v1 + v2
print(sum_vector)

# Q5: Create a vector with pattern: 1, 2, ..., 19, 20, 19, ..., 1
pattern_vector <- c(1:20, 19:1)
print(pattern_vector)

# Q6: Divide 2 vectors of integer type and length 3
v1 <- c(10L, 20L, 30L)
v2 <- c(2L, 4L, 5L)
div_vector <- v1 / v2
print(div_vector)

# Q7: Reverse the order of a given vector
vec <- c(5, 10, 15, 20, 25)
```

```
reversed_vec <- rev(vec)
print(reversed_vec)
```

28-Jan-2026

```
# Q1: Add, Delete, and Modify Elements in a Vector
vec <- c(10, 20, 30, 40)

# Add element
vec <- c(vec, 50)

# Delete element (remove 2nd element)
vec <- vec[-2]

# Modify element (change 3rd element)
vec[3] <- 100

print(vec)


# Q2: Generate 15 random numbers and find numbers greater than the average
set.seed(123)
rand_nums <- sample(1:100, 15)

avg <- mean(rand_nums)
greater_than_avg <- rand_nums[rand_nums > avg]

print(rand_nums)
print(avg)
print(greater_than_avg)


# Q3: Perform operations on a vector
v1 <- c(10, 20, 30)
v2 <- c(2, 4, 5)

# Sum, Mean, Max, Min
print(sum(v1))
print(mean(v1))
print(max(v1))
print(min(v1))

# Arithmetic operations
```

```
print(v1 + v2)
print(v1 - v2)
print(v1 * v2)
print(v1 / v2)


# Q4: Create a vector of colours of book cover and print its class
colors <- c("Red", "Blue", "Green", "Yellow")
print(colors)
print(class(colors))


# Q5: Create a list containing employee details
empid <- c(1, 2, 3, 4)
empname <- c("priyanka", "Isha", "rohan", "prakash")
numberofemp <- 4

emp_list <- list(empid = empid, empname = empname, numberofemp = numberofemp)

print(emp_list)
```

03-Feb-2026

```
# Q1: Creating a matrix of 2 rows and 3 columns
mat <- matrix(1:6, nrow = 2, ncol = 3)
print(mat)


# Q2: Creating an array of dim(3,3,2)
arr <- array(1:18, dim = c(3, 3, 2))
print(arr)


# Q3: Creating a factor
colors <- c("Red", "Blue", "Red", "Green", "Blue")
factor_colors <- factor(colors)
print(factor_colors)


# Q4: Create dataframe df = data.frame(Name, Language, Age)
Name <- c("Aman", "Riya", "Karan")
Language <- c("R", "Python", "Java")
```

```
Age <- c(20, 21, 22)

df <- data.frame(Name, Language, Age)
print(df)


# Q5: Create the data frame family_info
height <- c(132,151,162,139,166,147,122)
weight <- c(48,49,66,53,67,52,40)
gender <- c("male","male","female","female","male","female","male")
age <- c(12, 15, 18, 14, 20, 16, 10)

family_info <- data.frame(gender, height, weight, age)
print(family_info)


# Q6a: Create a vector of first 20 numbers
vec1 <- 1:20
print(vec1)


# Q6b: Create a vector 'class_students' with 10 student names
class_students <-
c("Aman","Riya","Karan","Priya","Rahul","Sneha","Amit","Neha","Rohit","Pooja")
print(class_students)


# Q6c: Create a vector to store age of these 10 students
student_age <- c(8,9,8,9,10,8,9,8,10,9)
print(student_age)


# Q7: Create a matrix of 4 flower names with 4 rows and 2 columns
flowers <- matrix(c("Rose","Lily","Tulip","Sunflower",
                    "Lotus","Daisy","Jasmine","Marigold"),
                  nrow = 4, ncol = 2)
print(flowers)


# Q8: Create an array of family member names with dim = c(4,3,2)
```

```
family <- array(c("A","B","C","D","E","F","G","H","I","J","K","L",
                 "M","N","O","P","Q","R","S","T","U","V","W","X"),
               dim = c(4,3,2))
print(family)


# Q9a: Create employee data frame with 7 employees
emp_name <- c("Aman","Riya","Karan","Priya","Rahul","Sneha","Amit")
emp_id <- c(101,102,103,104,105,106,107)
salary <- c(25000,30000,28000,27000,35000,26000,29000)
age <- c(25,26,27,24,28,23,29)
qualification <- c("BCA","MCA","BTech","BSc","MBA","BCom","MCA")

employee_df <- data.frame(emp_name, emp_id, salary, age, qualification)
print(employee_df)


# Q9b: Print the qualification column
print(employee_df$qualification)
```

11-Feb-2026

```
# Q1: Create matrix ‘a’ with 5 rows and 3 columns (filled by row)
a <- matrix(1:15, nrow = 5, ncol = 3, byrow = TRUE)
print(a)

# Q2: Create matrix ‘b’ using rbind with 5 rows and 3 columns
b <- rbind(c(1,2,3),
           c(4,5,6),
           c(7,8,9),
           c(10,11,12),
           c(13,14,15))
print(b)

# Q3: Create matrix ‘c’ using cbind with 3 rows and 5 columns
c <- cbind(c(1,2,3),
           c(4,5,6),
           c(7,8,9),
           c(10,11,12),
           c(13,14,15))
```

```
print(c)

# Q4: Transpose of matrix ‘a’
d <- t(a)
print(d)

# Q5: Matrix operations
r1 <- a + b
r2 <- a - b
r3 <- a / b
r4 <- b %*% c

print(r1)
print(r2)
print(r3)
print(r4)


# Q6: Create dataframe and perform operations
emp_id <- c(101,102,103,104,105)
name <- c("Aman","Riya","Karan","Priya","Rahul")
salary <- c(25000,30000,28000,27000,35000)
age <- c(25,26,27,24,28)

df <- data.frame(emp_id, name, salary, age)
print(df)

# Extract name and salary
print(df[, c("name","salary")])

# First two employees
print(df[1:2, ])

# Name and age of 3rd and 5th employee
print(df[c(3,5), c("name","age")])

# Add department column
df$department <- c("IT","HR","Finance","Sales","Admin")
print(df)
```

```
# Add 3 more employees
new_emp <- data.frame(
  emp_id = c(106,107,108),
  name = c("Sneha","Amit","Neha"),
  salary = c(26000,29000,31000),
  age = c(23,29,26),
  department = c("IT","HR","Finance")
)

df <- rbind(df, new_emp)
print(df)


# Q7: Create student dataframe and find dimensions
student_df <- data.frame(
  name = c("A","B","C","D","E"),
  age = c(18,19,20,21,22),
  height = c(160,165,170,155,168),
  weight = c(55,60,65,50,58)
)
print(student_df)
print(dim(student_df))


# Q8: Create dataframe of 8 students and perform filtering
students <- data.frame(
  roll_no = 1:8,
  name = c("A","B","C","D","E","F","G","H"),
  program =
c("BTech-EEE","BCom","MBA","BTech-EEE","MBA","BCom","BTech-EEE","MBA"),
  semester = c(1,2,1,3,2,1,4,3),
  email = paste0("student",1:8,"@gmail.com")
)
print(students)

# BTech-EEE students
print(students[students$program == "BTech-EEE", ])

# MBA students
print(students[students$program == "MBA", ])
```

```
# Rows and columns
print(dim(students))


# Q9: Merge two dataframes
df1 <- data.frame(ID = c(1,2,3), AGE = c(20,25,30))
df2 <- data.frame(ID = c(1,2,3), CITY = c("Delhi","Mumbai","Pune"))

merged_df <- merge(df1, df2, by = "ID")
print(merged_df)


# Q10: Find unique values in dataframe
dup_df <- data.frame(x = c(1,2,2,3,3,4))
print(unique(dup_df))


# Q11: Change column name
colnames(df1)[2] <- "Age_New"
print(df1)
```

13-Feb-2026

```
# Q1: Create a list, access elements, and merge lists
list1 <- list(a = 1:3, b = c("A","B","C"))
list2 <- list(c = TRUE, d = 10)

# Access elements
print(list1$a)
print(list1[[2]])

# Merge lists
merged_list <- c(list1, list2)
print(merged_list)


# Q2: Create a list containing strings, numbers, vectors and logical values
list_mixed <- list("Hello", 25, c(1,2,3), TRUE)
print(list_mixed)
```

```
# Q3: Create a list with vector, matrix and list and perform operations
vec <- c(1,2,3)
mat <- matrix(1:4, 2, 2)
inner_list <- list("X","Y")

my_list <- list(vector = vec, matrix = mat, sublist = inner_list)

# Print list
print(my_list)

# Access elements
print(my_list[[1]])
print(my_list[[3]])
print(my_list$matrix)
print(my_list[c(1,2)])


# Q4: Create a list of 3 vectors and one numeric value and perform operations
x <- list(c(1,2,3), c("a","b","c"), c(TRUE,FALSE,TRUE), 100)

# Print list
print(x)

# Print 2nd element
print(x[[2]])

# Print 2nd and 4th element
print(x[c(2,4)])

# Access 1st member of 2nd element
print(x[[2]][1])

# Update value
x[[2]][1] <- "ta"
print(x[[2]])


# Q5: Find elements of one list not in another list
```

```
list_a <- list(1,2,3,4)
list_b <- list(3,4,5)

result <- setdiff(unlist(list_a), unlist(list_b))
print(result)


# Q6: Find row and column index of max and min value in a matrix
mat2 <- matrix(c(5,2,9,1,7,3), nrow = 2)

max_pos <- which(mat2 == max(mat2), arr.ind = TRUE)
min_pos <- which(mat2 == min(mat2), arr.ind = TRUE)

print(mat2)
print(max_pos)
print(min_pos)


# Q7: Create two matrices and perform multiplication
A <- matrix(1:6, nrow = 2)
B <- matrix(7:12, nrow = 3)

result_mat <- A %*% B
print(A)
print(B)
print(result_mat)
```

25-Feb-2026

```
# Q1: Print Hello World 4 times using repeat loop (count starts at 2)
count <- 2
repeat {
  print("Hello World")
  count <- count + 1
  if(count > 5) break
}


# Q2: Print Hello While Loop 5 times using while loop (count starts at 2)
count <- 2
```

```
while(count <= 6) {
  print("Hello While Loop")
  count <- count + 1
}


# Q3: Print first 4 alphabets using for loop
for(i in letters[1:4]) {
  print(i)
}


# Q4: Illustrate break statement
for(i in 1:10) {
  if(i == 5) break
  print(i)
}


# Q5: Illustrate next statement
for(i in 1:10) {
  if(i == 5) next
  print(i)
}


# Q6: Built-in functions usage
# Sequence from 32 to 44
seq_nums <- seq(32, 44)
print(seq_nums)

# Mean from 25 to 82
print(mean(25:82))

# Sum from 41 to 68
print(sum(41:68))


# Q7: Functions
# Squares of first 6 numbers (with argument)
```

```
square_func <- function(n) {
  print((1:n)^2)
}
square_func(6)

# Squares of first 5 numbers (without argument)
square_no_arg <- function() {
  print((1:5)^2)
}
square_no_arg()

# Function a*b + c
calc <- function(a, b, c) {
  return(a*b + c)
}
print(calc(2,3,4))          # by position
print(calc(a=2, b=3, c=4))   # by name

# Function with default arguments
mul_default <- function(a=2, b=3) {
  return(a*b)
}
print(mul_default())
print(mul_default(5,6))


# Q8: Multiplication table using for loop
num <- 5
for(i in 1:10) {
  print(paste(num, "x", i, "=", num*i))
}


# Q9: Count even numbers in a vector
vec <- c(1,2,3,4,5,6,7,8,9,10)
even_count <- sum(vec %% 2 == 0)
print(even_count)


# Q10: Print square root of integers 1 to 10
```

```
print(sqrt(1:10))


# Q11: Function to calculate power
power_func <- function(a, b) {
  return(a^b)
}
print(power_func(2,3))


# Q12: Fahrenheit to Celsius
fahrenheit_to_celsius <- function(f) {
  return((f - 32) * 5/9)
}
print(fahrenheit_to_celsius(98.6))


# Q13: Celsius to Kelvin
celsius_to_kelvin <- function(c) {
  return(c + 273.15)
}
print(celsius_to_kelvin(25))


# Q14: Area of a circle
area_circle <- function(r) {
  return(pi * r^2)
}
print(area_circle(5))
```

2-March-2026

```
# Q1: Classify Sepal.Length in iris dataset using for loop
data(iris)

result <- c()
for(i in 1:nrow(iris)) {
  if(iris$Sepal.Length[i] > 5) {
    result[i] <- "greater than 5"
  } else {
```

```
    result[i] <- "lesser than 5"
  }
}
print(result)


# Q2: Use seq() and rep() to generate sequences
print(seq(1, 10, by = 2))
print(rep(c(1,2,3), times = 3))
print(rep(1:3, each = 2))


# Q3: Compute mean values
vec <- c(12,7,3,4.2,18,2,54,-21,8,-5)

# Mean
print(mean(vec))

# Trimmed mean (remove 3 values from each end)
print(mean(vec, trim = 3/length(vec)))

# Mean with NA
vec_na <- c(12,7,3,4.2,18,2,54,-21,8,-5,NA)
print(mean(vec_na, na.rm = TRUE))


# Q4: Compute median
vec1 <- c(12,7,3,4.2,18,2,54,-21,8,-5)
vec2 <- c(2,13,8,7,15,3,9)

print(median(vec1))
print(median(vec2))


# Q5: Compute mode of a vector
vec_mode <- c(2,1,2,3,1,2,3,4,1,5,5,3,2,3)
mode_val <- names(sort(table(vec_mode), decreasing = TRUE))[1]
print(mode_val)
```

```
# Q6: Student age analysis
age <- c(18,18,20,19,22,21,22,23,22,20,19,20,19,22,18)

# Mean, Median, Mode
print(mean(age))
print(median(age))
mode_age <- names(sort(table(age), decreasing = TRUE))[1]
print(mode_age)

# Median age under 22
age1 <- age[age < 22]
print(median(age1))

# Add two students age 23
age_new <- c(age, 23, 23)
print(mean(age_new))
print(median(age_new))
mode_new <- names(sort(table(age_new), decreasing = TRUE))[1]
print(mode_new)


# Q7: typeof(), length(), class()
x <- 10
y <- c(1,2,3)
z <- "Hello"

print(typeof(x)); print(length(x)); print(class(x))
print(typeof(y)); print(length(y)); print(class(y))
print(typeof(z)); print(length(z)); print(class(z))


# Q8: Basic plotting
plot(1:10, main="Simple Plot", xlab="X", ylab="Y")
hist(vec, main="Histogram", xlab="Values")


# Q9: Recursive functions
# Factorial
fact <- function(n) {
  if(n == 0) return(1)
```

```
  else return(n * fact(n-1))
}
print(fact(5))

# Fibonacci
fib <- function(n) {
  if(n <= 1) return(n)
  else return(fib(n-1) + fib(n-2))
}
print(fib(6))


# Q10: Create list and perform slicing
my_list <- list(a = 1:5, b = c("x","y","z"), c = TRUE)

print(my_list[1])
print(my_list[1:2])
print(my_list$b)
```

6-March-2026

```
# Q1: Create a data frame and perform operations
df <- data.frame(name=c("A","B","C"), age=c(20,21,22), salary=c(20000,25000,30000))
print(df)

# Add column
df$dept <- c("IT","HR","Finance")

# Remove column
df$salary <- NULL

# Add row
df <- rbind(df, data.frame(name="D", age=23, dept="Sales"))

# Remove row
df <- df[-2, ]

# Access element
print(df$name)
```

```
print(df[1, ])
print(df[, "age"])


# Q2: Expand a data frame
df2 <- data.frame(id=1:3, value=c(10,20,30))
df2 <- cbind(df2, new_col=c(100,200,300))
df2 <- rbind(df2, data.frame(id=4, value=40, new_col=400))
print(df2)


# Q3: Merge data frames from iris dataset
data(iris)
Df1 <- iris[1:50, ]
Df2 <- iris[51:100, ]

merged_df <- rbind(Df1, Df2)
print(head(merged_df))


# Q4: apply(), lapply(), sapply()
mat <- matrix(1:9, nrow=3)
print(apply(mat, 1, sum))

lst <- list(a=1:3, b=4:6)
print(lapply(lst, sum))
print(sapply(lst, sum))


# Q5: Plot single point
plot(3,5, main="Single Point", xlab="X", ylab="Y")


# Q6: Plot multiple points
x <- c(1,2,3,4,5)
y <- c(2,4,6,8,10)
plot(x,y)


# Q7: Plot points with different colors and symbols
```

```
plot(x,y, col=c("red","blue","green","purple","orange"), pch=1:5)


# Q8: Plot given points with labels and title
x1 <- c(2,4,6,8)
y1 <- c(4,6,8,10)
plot(x1,y1, main="Custom Points", xlab="X-axis", ylab="Y-axis")


# Q9: Draw straight line through given points
x2 <- c(1,2,3,4)
y2 <- c(2,4,6,8)
plot(x2,y2, type="l")


# Q10: Line graph using vectors
x3 <- c(1,2,3,4,5)
y3 <- c(5,4,3,2,1)
plot(x3,y3, type="l")


# Q11: Line graph for y = 2x + 1
x4 <- 1:10
y4 <- 2*x4 + 1
plot(x4,y4, type="l")


# Q12: Line with different types and colors
plot(x4,y4, type="l", col="blue", lty=2)


# Q13: Line graph with grid
plot(x4,y4, type="l")
grid()


# Q14: Recursive functions
fact <- function(n) {
  if(n==0) return(1)
  else return(n*fact(n-1))
```

```
}
print(fact(5))

fib <- function(n) {
  if(n<=1) return(n)
  else return(fib(n-1)+fib(n-2))
}
print(fib(6))


# Q15: List and slicing
my_list <- list(a=1:5, b=c("x","y"), c=TRUE)
print(my_list[1])
print(my_list[1:2])
print(my_list$b)


# Q16: Function to summarize each column in a data frame
summary_func <- function(df) {
  return(summary(df))
}
print(summary_func(iris))


# Q17: Use rbind() and cbind() to expand data
df3 <- data.frame(a=1:3, b=4:6)
df3 <- cbind(df3, c=7:9)
df3 <- rbind(df3, data.frame(a=4, b=7, c=10))
print(df3)
```

8th-April-2026

```
# Q1: Plot Product A and Product B sales
productA <- c(20, 35, 40, 55, 70, 90)
productB <- c(15, 25, 30, 45, 60, 80)
months <- 1:6

plot(months, productA, type="l", col="blue", lty=1,
     xlab="Months", ylab="Sales", main="Product Sales Comparison")
```

```
lines(months, productB, col="red", lty=2)
legend("topleft", legend=c("Product A","Product B"),
       col=c("blue","red"), lty=c(1,2))


# Q2: Scatter plot of height vs weight
height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175)
weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70)

plot(height, weight, col="blue", pch=19,
     xlab="Height (cm)", ylab="Weight (kg)", main="Height vs Weight")
legend("topleft", legend="Students", col="blue", pch=19)


# Q3: Pie chart of sales distribution
sales <- c(25, 30, 20, 25)
regions <- c("North", "South", "East", "West")

colors <- c("red","blue","green","yellow")
pie(sales, labels=regions, col=colors, main="Sales Distribution")
legend("topright", legend=regions, fill=colors)


# Q4: Monthly temperature line chart
months <- 1:12
temperature <- c(15, 18, 22, 28, 32, 35, 34, 33, 30, 25, 20, 16)

plot(months, temperature, type="l", col="red", lty=2,
     xlab="Months", ylab="Temperature", main="Monthly Temperature")


# Q5: Bar chart of sales by region
sales <- c(120, 150, 100, 130)
regions <- c("North", "South", "East", "West")

bar_colors <- c("red","blue","green","orange")
barplot(sales, names.arg=regions, col=bar_colors,
        main="Sales by Region", xlab="Region", ylab="Sales")
legend("topright", legend=regions, fill=bar_colors)
```

```
# Q6: Box plot for Class A and Class B
classA <- c(65, 70, 75, 80, 85, 90)
classB <- c(60, 62, 68, 72, 78, 82)

boxplot(classA, classB, col=c("blue","green"),
        names=c("Class A","Class B"),
        main="Class Scores Comparison", xlab="Class", ylab="Scores")


# Q7: Scatter plot with different symbols for groups
height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175)
weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70)

group <- c("M","F","M","F","M","F","M","F","M","F")
pch_vals <- ifelse(group=="M", 16, 17)
col_vals <- ifelse(group=="M", "blue", "red")

plot(height, weight, pch=pch_vals, col=col_vals,
     xlab="Height", ylab="Weight", main="Height vs Weight (Group-wise)")
legend("topleft", legend=c("Male","Female"),
       col=c("blue","red"), pch=c(16,17))


# Q8: Pie chart with legend
sales <- c(25, 30, 20, 25)
regions <- c("North", "South", "East", "West")

colors <- c("purple","cyan","orange","pink")
pie(sales, labels=regions, col=colors, main="Sales Distribution")
legend("topright", legend=regions, fill=colors)
```

16-April-2026

```
# Q1: Exploratory Data Analysis of Big Mart Dataset
# (Assuming file: bigmart.csv)
data <- read.csv("bigmart.csv")

print(head(data))
print(summary(data))
```

```
print(str(data))


# Q2: Graphical Representation (Univariate & Bivariate) - Big Mart
# Histogram
hist(data$Item_Outlet_Sales, main="Sales Distribution", xlab="Sales")

# Boxplot
boxplot(data$Item_Outlet_Sales, main="Boxplot of Sales")

# Scatter Plot
plot(data$Item_MRP, data$Item_Outlet_Sales,
     xlab="MRP", ylab="Sales", main="MRP vs Sales")

# Bar Plot
barplot(table(data$Item_Fat_Content), main="Fat Content Distribution")


# Q3: Graphical Representation of Train Dataset
train <- read.csv("train.csv")

# Univariate
hist(train$Item_Outlet_Sales)
boxplot(train$Item_Outlet_Sales)

# Bivariate
plot(train$Item_MRP, train$Item_Outlet_Sales)


# Q4: Linear Model using multiple variables
# (Example with available columns)
model <- lm(Item_Outlet_Sales ~ ., data=data)
print(summary(model))


# Q5: Import/Export Data
# Read CSV
df <- read.csv("data.csv")

# Write CSV
```

```
write.csv(df, "output.csv")

# Read Excel
library(readxl)
excel_data <- read_excel("data.xlsx")

# Read JSON
library(jsonlite)
json_data <- fromJSON("data.json")


# Q6: Data Cleaning
# Remove NA
clean_data <- na.omit(data)

# Check NA
print(is.na(data))

# Using tidyr
library(tidyr)
clean_data2 <- drop_na(data)


# Q7: Prediction Model using lm()
model <- lm(Item_Outlet_Sales ~ Item_MRP + Item_Visibility, data=data)

# Summary
print(summary(model))

# Residual Plot
plot(model$residuals, main="Residual Plot")


# Q8: Astrology + Panchang Analysis (Basic Structure)
# Sample dataset (dummy)
astro <- data.frame(
  planet=c("Mars","Saturn","Rahu"),
  dosha=c("Mangal","Shani","Kaal"),
  health_score=c(70,50,40)
)
```

```
# Simple analysis
print(table(astro$dosha))
plot(astro$health_score, main="Health vs Dosha")


# Q9: Air & Water Pollution Analysis
pollution <- data.frame(
  AQI=c(100,120,90,150),
  pH=c(7,6.5,7.2,6.8),
  turbidity=c(3,4,2,5)
)

# Visualization
plot(pollution$AQI, type="l", main="AQI Trend")

# Prediction model
model_poll <- lm(AQI ~ pH + turbidity, data=pollution)
print(summary(model_poll))


# Q10: Stock Price Prediction
stock <- data.frame(
  price=c(100,105,110,120),
  volume=c(2000,2500,2300,2600)
)

plot(stock$price, type="l", main="Stock Price Trend")

model_stock <- lm(price ~ volume, data=stock)
print(summary(model_stock))


# Q11: Gender Equality Analysis
gender <- data.frame(
  education=c(60,70,80),
  employment=c(50,65,75),
  income=c(40,55,70)
)
```

```
# Visualization
barplot(as.matrix(gender), beside=TRUE, main="Gender Metrics")

# Model
model_gender <- lm(income ~ education + employment, data=gender)
print(summary(model_gender))
```